



Agents That Cost Bytes

Zach Kelling

Lux Industries

2026

The number. An agent costs 601 bytes of heap while it is live and 477 bytes on disk while it is not [measured]. A million live agents occupy 573.0 MB, spawn in 125 ns each, and wake in 107 ms — 107 ns apiece [measured]. A million dormant agents occupy 455 MB and are written at 309,789 per second; resuming one in process costs 0.034 ms [measured]. These are one pass of `hanzoai/cloud bench` on one laptop, and the host is named beside them because an earlier version of this table was not reproducible: re-running the goroutine lane on a slower M1 Max returned 452 ns to spawn against a published 187 and 347 ns to wake against 114 [verified]. The figures stand; the machine is now part of the figure.

The problem, in their numbers. Morpheus sells agents and does not run any. Seven repositories in the organisation carry “agent” in the name [verified 2026-09-17]. Four — `morpheus-open-agents`, `morpheus-coding-agent`, `morpheus-community-agent`, `morpheus-knowledge-agent` — are forks of vercel-labs templates, every one of them last touched 2026-04-21 [verified]. Two more are dormant: `agent-computer-use` (2025-07-22) and `SmartAgents`, itself a fork, last touched 2024-05-28 [verified]. The seventh, `MySuperAgent`, is the live one (2026-09-08 in our clone) and is a Next.js web application: its own documentation describes “client-side agent orchestration and execution” over “local SQLite for client-side data storage” (`CLAUDE.md:17-18`) [verified]. None of the seven is a server-side agent runtime.

The sandbox is the sharper case. `MySuperAgent` exposes a tool whose description reads “*Execute Python code in a safe sandbox environment*” (`code-tools.ts:6`). The implementation executes nothing. It returns a string, above a comment reading “In a real implementation, you’d use a sandbox like Pyodide or a secure container” (`code-tools.ts:17-19`) [verified]. An agent marketplace whose code executor is a stub has not deferred the hard part; it has not reached it. Nothing settles on work that never ran, which is the same meter problem ADOPT-07 names, arriving one layer up.

What a comparison is worth. Numbers against competitors are reported here in three columns, because two is how a wrong one survives. *Ours* is what this pass measured. *Theirs* is a figure attributed to somebody else and not re-run — their protocol, their hardware, their claim. *Checked* is what their library did when we ran it here.

The third column is the one that matters. This lane previously reported a V8 context at 0.15 ms against a cited 2.79 ms and called the difference seventeenfold. Those are not the same primitive: `vm.createContext` makes a fresh global inside an isolate that is already running, and it is cheap precisely because it shares the heap — the one thing an isolate does not do. Running the library the 2.79 ms is attributed to, `isolated-vm 7.0.1`, on this laptop, an isolate costs 0.35 ms and 1.00 MiB [checked]. Still under the figure cited at us, by 4.7× rather than 17×, and a measurement instead of an argument. The upstream commit that corrected it is titled “Measure the isolate instead of citing one”.

So the megabyte is V8’s, not a vendor’s: any isolate costs about that, including one of ours. The honest comparison is 601 bytes for a live agent against 1.00 MiB for an isolate [checked], and a 125 ns spawn against 0.35 ms [checked]. A comparison is worth printing only when both sides are the same primitive.

The same discipline cuts against us elsewhere, and is printed anyway. `hanzo-vm` cold boots in 309 ms and resumes from a checkpoint in 311 ms [measured], against E2B under 200 ms, Modal near a second, and Cloudflare one to three seconds [attributed]. On cold boot E2B is ahead. The density claim is about agents, not about virtual machines, and stating where it does not hold is what makes the rest checkable.

The fix, named to a repo. A Morpheus provider runs `Morpheus-Lumerin-Node/proxy-router` and nothing in it changes. Alongside it, `hanzoai/cloud` supplies the runtime the agent repositories assume and do not contain: agents that persist as bytes rather than processes, an execution surface that actually executes, and a meter that counts what ran. Extensions host in process from the same pass — `wazero` instantiates a WASM module in 7.8 μs and calls into it in 20 ns; `goja` costs 2.4 μs per JavaScript VM and 740 ns per warm eval; `gpython` costs 25.0 μs per context [measured]. An agent that needs a language runtime does not need a container for it.

The density is the economic argument. At 601 bytes live and 477 dormant, a provider hosts agent populations per box that a per-isolate or per-container model prices out of reach: the isolate at 1.00 MiB [checked] is roughly $1,700\times$ the live cost, before any container overhead. Capacity, work and identity meter as three quantities, and work pays only on counts the consumer also signed — so a faked session earns zero, and the settlement problem ADOPT-07 describes for inference is solved the same way for agents.

The outcome, projected. Assumptions stated. Morpheus keeps its contracts, its node, and its token. It gains a runtime under the seven agent repositories that currently have none, a code executor that is not a stub, and a meter that makes agent work settleable on the same rails as inference. The reciprocal case is ADOPT-10, and the open-source path ADOPT-15: the runtime is open, so a provider who adopts it reads every line of what meters them. What Morpheus supplies in return is demand and a settlement layer already live on Base — which is the loop, not a favour.

What would falsify this. A published Morpheus agent runtime that executes untrusted code server-side, with its own density and isolation figures on a named host. None was found across the seven agent repositories on 2026-09-17, when the organisation stood at 63 public repositories [verified]. That is a search of the named surface, not a proof of absence across all 63. If one ships, this brief should be re-run against it rather than restated.